Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп’ютерних наук та інформаційних технологій
Кафедра систем штучного інтелекту



Звіт

до лабораторної роботи № 4

з дисципліни

Проектування інформаційних систем

Виконав:
студент КН-315
Лозовий О.А.

Львів – 2023

Функціональні вимоги:

1. Безпека даних
    a. Система повинна використовувати сучасні протоколи шифрування.
    b. Система має двофакторну аутентифікацію.
    c. Всі дані, які передаються та зберігаються, повинні бути зашифровані.
2. Робота з документами
    a. Система повинна автоматично перевіряти подані документи на ідентичність.
    b. Інтеграція з органами, що видають документи, для швидкої і надійної перевірки.
3. Оцінка кредитного ризику
    a. Система повинна мати алгоритми для оцінки кредитної здатності особи.
    b. Система використовуватиме машинне навчання для аналізу фінансової історії, перевірки доходів та інших показників.
    c. Система повинна використовувати наявні бази фінансових історій користувачів.
4. Взаємодія з клієнтами
    a. Надсилання нотифікацій про стан заявки.
    b. Користувач має мати можливість отримати доступ до інформації через особистий кабінет.
    c. Користувач має мати можливість отримати доступ до перегляду стану заявки, історії платежів, та іншої важливої інформації.
5. Платіжні системи
    a. Інтеграція з платіжними системами для здійснення транзакцій.
    b. Можливість оплатити різними засобами платежу.
6. Аудит та звітність
    a. Автоматична генерація звітів для внутрішнього аудиту та регуляторних органів.

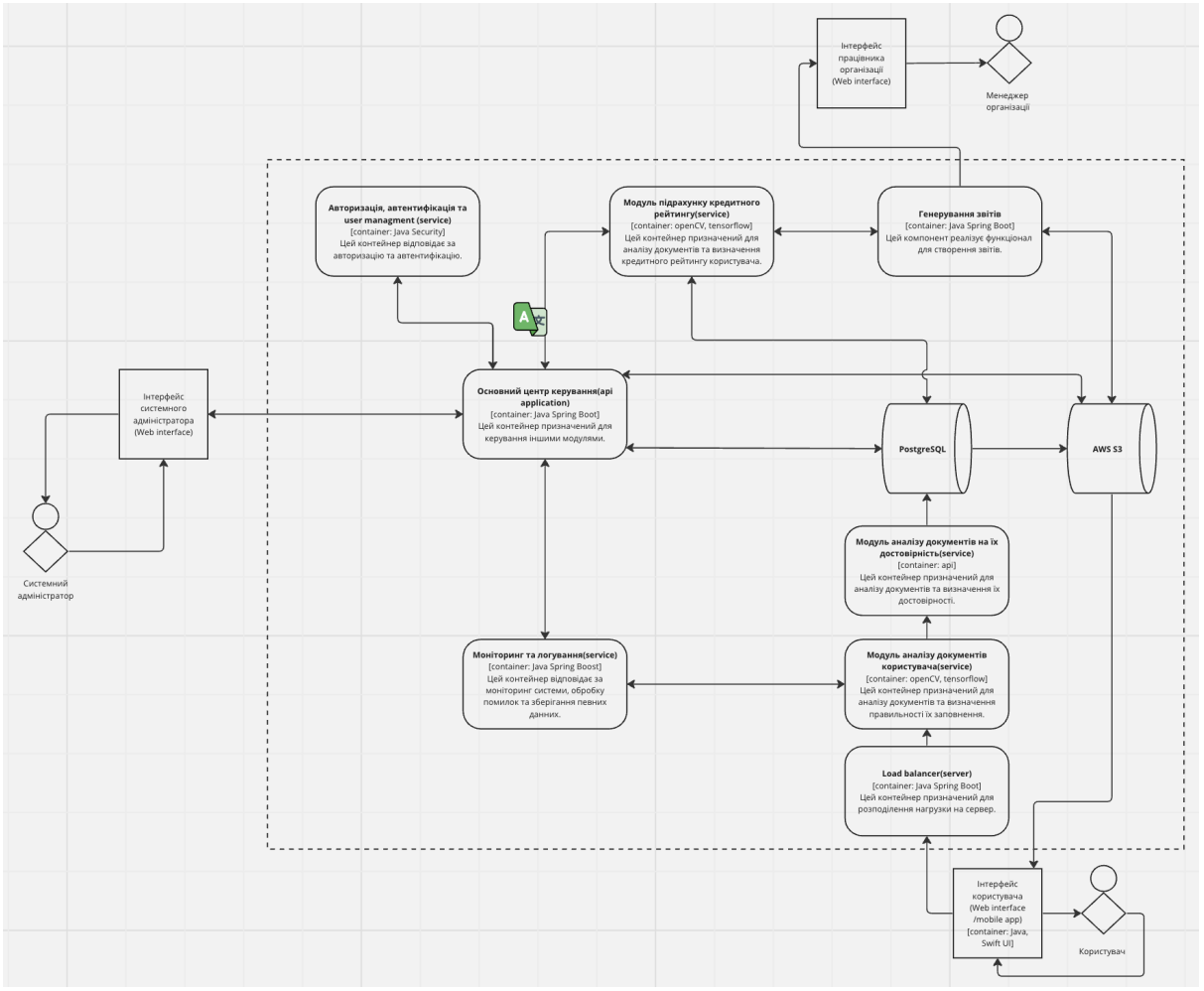
Нефункціональні вимоги:

1. Уся персональна інформація повинна зберігатись за стандартами GDPR.
2. Підтримка платежів через банківські картки, електронні гаманці та інші засоби платежу.
3. Створені звіти повинні відповідати стандартам і вимогам регуляторів.

Architecturally Significant Requirements:

1. Збереження даних
    a. Збереження звітів вимагає таку базу даних, яка легко може збільшуватись разом зі збільшенням кількості звітів.
    b. Дані користувачів також необхідно зберігати (у зашифрованому вигляді).
    c. Для зберігання резервних копій моделі(на випадок збою), необхідно виділити місце на сервері.
2. Система повинна підтримувати можливість збільшення кількості користувачів
    a. Збільшення користувачів приведе до потреби більшої бази даних, яка може вмістити більшу кількість користувачів.
    b. Також при збільшенні кількості запитів, можливі збої в роботі, тому необхідно обмежити трафік при великих навантаженнях.
3. Інтеграції
    a. Для інтеграції з органами, що видають документи, необхідно створити API, або ж інтегрувати зовнішнє API.
    b. Для інтеграції машинного навчання, необхідно створити модель, натренувати її та перевірити точність її роботи.
4. Мультиплатформність
    a. Необхідно створити мобільний додаток для IOS і Android.
    b. Необхідно створити веб-інтерфейс для інших платформ.

Діаграма контейнерів:



Для розробки свого проекту я вирішив використовувати мікросервісну архітектуру, оскільки вважаю, що вона найбільш відповідна в даному випадку. Мікросервіси дозволяють ефективно розділити функціональність системи, дозволяючи розробникам працювати паралельно над різними компонентами сервісу, що прискорює виконання проекту. Крім того, вони забезпечують можливість незалежного масштабування окремих компонентів системи, що дозволяє оптимізувати використання ресурсів та швидко реагувати на зміни у навантаженні.

Branching strategy

Gitflow:

- Основні гілки: master, develop.
- Додаткові гілки: feature (для роботи над новими функціями), release (для підготовки до релізу), hotfix (для виправлення невеликих помилок у виробничому середовищі).

- Переваги: Ця стратегія гнучка і надійна для проектів середнього та великого розміру. Вона дозволяє розробникам паралельно працювати над різними функціями, і відокремлює розробку від готовності до релізу. Також дозволяє забезпечити проект від непередбачуваних помилок(поспішних релізів).

CI/CD (Основний центр керування):

1. Кодування та Розробка:

Створення гілок:

a. Розробники створюють відгалуження (branches) для кожної функції або задачі.
   i. Видаються відповідні права доступу до гілок.
b. Використовуються основні гілки, такі як master, develop, і окремі гілки для роботи над функціоналом(feature, release, hotfix).
   i. Для кожної гілки створюється файл з її описом.

Імплементація коду:

c. Розробники створюють новий функціонал.
   i. Реалізуються такі складові, як Аутентифікація, Керування базами даних.
   ii. Реалізується функціонал для Налаштування системи, Моніторингу та аналізу її продуктивності.
   iii. Реалізується функціонал для отримання звітів про стан системи, її навантаження, тощо.
   iv. Використовуватимуться юніт-тести та інтеграційні тести, а також мокування для імітації зовнішніх залежностей. Для документування можна використовувати docstrings, а для створення загальної документації важливо створити README.md файл, який міститиме загальну інформацію про проект, включаючи використовувані бібліотеки, структуру проекту та основні концепції.
d. Використовується система контролю версій (Git).

Потрібно використовувати інструменти статичного аналізу, такі як SonarQube, для виявлення потенційних проблем та покращення якості коду.

2. Continuous Integration (CI):

Основна мета CI - забезпечити швидке та автоматизоване тестування та злиття коду, щоб виявляти та виправляти конфлікти та помилки раніше в процесі розробки.

Commit та Pull Requests:

e. Розробники роблять коміти змін у своїх гілках.
f. Відправляють pull requests (PR) для об'єднання своїх гілок з основною.

Налаштування CI-сервера:

g. Для CI потрібно конфігурувати Jenkins для автоматичного виконання тестів

h. Збірки проекту при кожному коміті в гілку feature

Скрипти автоматизації:

i. Створюємо скрипти для автоматизації завдань, таких як встановлення залежностей, виконання тестів та генерація документації.

Матриця тестування:

j. Задаємо матрицю тестування для різних конфігурацій, таких як версії Python, різні версії бібліотек, та операційні системи.

Code Review:

k. Програміст створює запит на внесення змін (pull request), в якому включає свої зміни з конкретної гілки в основну гілку (наприклад, з гілки "feature" в гілку "main").
l. Інші учасники команди аналізують код та залишають коментарі в PR, вказуючи на можливі виправлення, покращення або стилістичні питання.
m. Проводиться технічне рев'ю, під час якого перевіряється правильність виконання функціональних вимог, відповідність кодовому стилю та використання кращих практик.
n. Розробник вносить необхідні корективи та відповідає на коментарі у PR.
o. Цей ітеративний процес триває, поки код не отримає погодження. Після успішного рев'ю та внесення необхідних змін, pull request затверджується та зливається з основною гілкою.

3. Continuous Delivery (CD):

Continuous Delivery (CD) важливий для неперервного та ефективного випуску програмного забезпечення. Його основна мета - автоматизація процесів від розробки до випуску.

p. Deployment в тестове середовище:
    i. Код розгортається в тестовому середовищі для остаточного тестування перед впровадженням в продакшн.
q. Staging:
    i. Якщо тестове середовище успішно пройшло, код переходить в staging для тестування в умовах, якнайбільш близьких до продакшн.
    ii. Створюємо навантаження на систему для перевірки Stability Gate і Performance Gate
r. Quality Gates:
    i. Якщо всі тести пройдені, необхідно перевірити Quality Gates
    ii. Використовуватимемо Jenkins Pipelines для встановлення Quality Gates, щоб автоматично перевіряти покриття коду, якість коду та виявляти відсутність нових помилок перед розгортанням.

s. Логування та Моніторинг:

    i. ELK Stack використовуватимемо для централізованого збору та аналізу логів з різних компонентів сервісу

    ii. Завдяки логуванню та моніторингу, забезпечується швидке виявлення можливих проблем та відповідне реагування на них. Автоматизоване сповіщення та аналіз логів допомагають зменшити час відкликання на інциденти та поліпшити загальну доступність сервісу.

Коли всі тести, Quality Gates на тестовому та staging середовищах успішно пройдені, код може бути випущений в продакшн. Для цього використаємо Rollback Strategy(Забезпечення можливості швидкого відновлення до попереднього стану.)

t. Deployment в продакшн:
    i. Використовуємо Continuous Delivery(Зберігає фінальне рішення про випуск виробничої версії у руках команди

розробників чи операторів), оскільки нам важливо забезпечити мінімальну кількість помилок на продакшині.

4. Моніторинг та Підтримка:

Monitoring та Rollback:

u. Для забезпечення ефективного моніторингу продукції та оперативного виявлення проблем в реальному часі використовуватимемо AppDynamics. AppDynamics надає комплексний підхід до APM (Application Performance Monitoring), що забезпечує детальний інсайт у функціонування програм, виявлення проблем та оптимізацію продуктивності.

v. Після випуску в продакшн важливо вести моніторинг системи на випадок появи проблем.

w. Якщо виявлено проблеми, використовується механізм автоматичного відкочування (rollback).

Continuous Monitoring та Feedback:

x. Забезпечення постійного моніторингу та отримання зворотного зв'язку для подальшого вдосконалення процесу CI/CD.

y. Створення системи відстеження звернень користувачів та планування випусків є ключовим елементом у забезпеченні ефективної підтримки та розвитку програмного продукту.

Версіонування:

z. Для версіонування використовуватимемо Semantic Versioning (SemVer). Semantic Versioning - стандарт позначення версій, де номер версії визначається у форматі MAJOR.MINOR.PATCH, відображаючи значущість змін.

Висновок: У ході лабораторної роботи була проведена детальна розробка функціональних та нефункціональних вимог до інформаційної системи, спрямованої на автоматизацію процесів надання кредитів. Основна увага була приділена забезпеченню безпеки даних, автоматизованій перевірці документів, оцінці кредитного ризику та взаємодії з клієнтами.

На основі вимог була розроблена архітектурно значуща стратегія гілкування (Gitflow), яка дозволяє ефективно керувати процесом розробки та випуску нового функціоналу. Також було визначено ключові етапи CI/CD, включаючи імплементацію коду, використання гілок, взаємодію з кодовим рецензуванням, автоматизоване тестування та етапи випуску виробничої версії.

Важливими аспектами роботи над системою є забезпечення безпеки даних відповідно до стандартів GDPR, інтеграція з різними платіжними системами, створення мультиплатформного інтерфейсу для користувачів, а також використання механізмів моніторингу та зворотного зв'язку для подальшого вдосконалення системи.

Отже, розроблені вимоги та стратегії реалізації дозволяють створити надійну та ефективну інформаційну систему для управління процесами надання кредитів, забезпечуючи високий рівень безпеки, швидкість впровадження нового функціоналу та стабільність роботи виробничого середовища.